

업무 메모 한 장이 프로그램이 되기까지.

AI 기반 Vibe Coding · 초급

2026. 09. 29. 화 16:00-17:00

말로 요청하고, 실행하고,
업무 기준으로 확인하는 첫 한 시간.



사람은 기준을 정하고

AI는 구현을 돕습니다

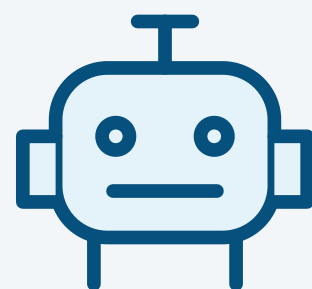
오늘은 외우지 않아도 됩니다.

용어보다 먼저, 일이 흘러가는 모습을 이해합니다.



이런 게 있구나

흐름을 보고, 모르는 것은 그때 물어봅니다.



일단 만들어줘

작은 결과를 부탁하고 실제로 실행해 봅니다.



여기는 고쳐줘

내 업무의 기준으로 확인하고 수정합니다.

용어를 외우는 대신, **만들고 확인하는 흐름**을 가져갑니다.

AI에게 없는 것은 당신이 아는 업무입니다.

개발에 참여할 이유는 코딩 실력보다 업무를 판단하는 힘에 있습니다.



기한이 없는 요청
어떻게 처리할까요?

날짜 추측

확인 필요 ✓

업무를 아는 사람의 판단

무엇이 필요한지, 예외를 어떻게 처리해야 하는지 알려줍니다.

AI의 실행력

그 기준을 코드와 화면으로 구현하고 실행합니다.

검증의 기준도 사람에게

프로그램이 실행되는 것과 업무에 맞는 것은 따로 확인합니다.

답변이 파일로 이어지는 순간.

설명을 받는 대화에서, 실제 작업 공간을 다루는 대화로.

설명 중심의 챗봇



사용자가 답변을 파일에 옮기고 실행합니다.

도구를 쓰는 코딩 에이전트

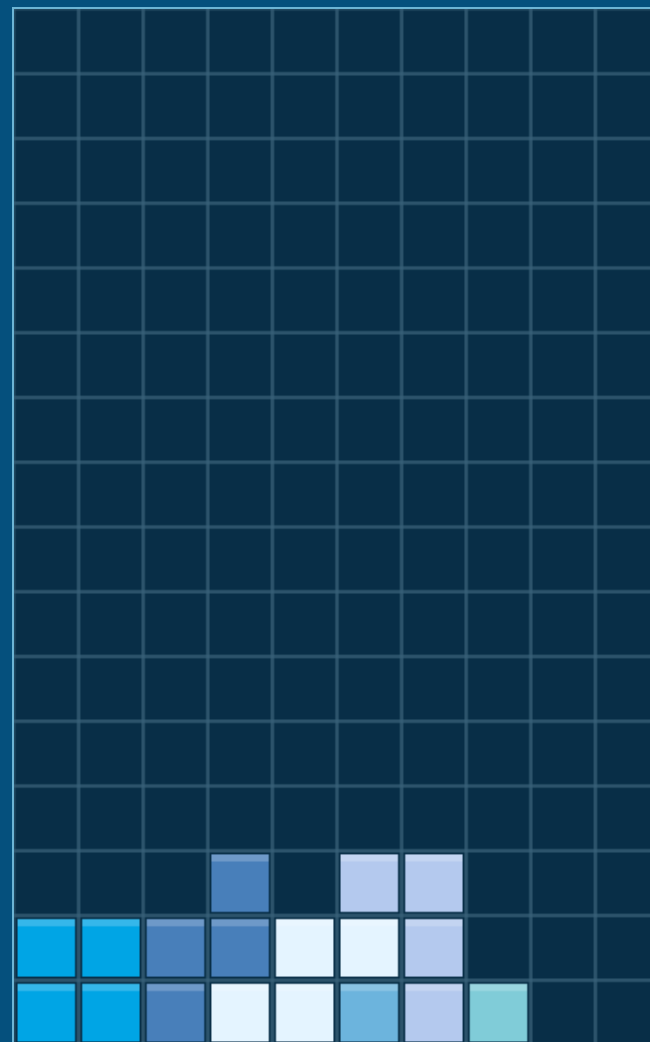


연결된 도구로 파일과 실행 결과를 직접 다룹니다.

가능한 행동은 연결된 도구와 권한에 따라 달라집니다. 화면·마우스 제어는 별도 도구가 필요합니다.

말로 바꾸는 감각부터.

게임을 보며 “조금 느리게, 점수는 더 크게”라고 요청한다고 생각해 보세요.



지운 줄

1

자동 배치 중

“속도를 낮춰줘.”

자동 플레이 속도 2

“점수를 더 잘 보이게 해줘.”

점수 강조

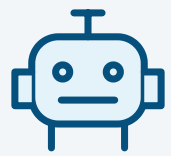
새 게임

원하는 변화를 말하고, 눈앞의 결과로 확인합니다. 코드 문법을 알아야만 변경을 요청할 수 있는 것은 아닙니다.

준비된 HTML 게임 · 현재 조작은 로컬 코드 실행이며 AI 실시간 생성이 아닙니다.

오늘의 한 시간은 이 메모를 따라갑니다.

가능성을 보고 → 구조를 이해하고 → 만들고 → 사내 AI에 연결합니다.



08

분 · 가능성

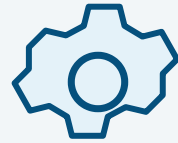
챗봇에서
에이전트로



15

분 · 기초

화면에서
실행 환경까지



15

분 · 제작

요청하고
확인하고 수정



09

분 · 준비

개발도구와
실행 방법



08

분 · 사내 연결

P-GPT와
API의 역할



05

분 · Q&A

내 업무로
가져갈 질문

설치를 동시에 따라 하는 시간이 아닙니다. 제작 시연을 보면서 각 도구가 맡은 역할을 연결해 봅니다.

화면 뒤를 펼쳐 보면.

우리가 보는 업무 도구는 역할이 다른 부분들이 연결된 결과입니다.



프론트엔드

보이고 반응하는 화면

API

요청·응답의 약속

백엔드

업무 규칙을 실행

DB

필요한 자료를 저장

모든 도구에 서버나 DB가 필요한 것은 아닙니다. 작은 HTML 도구는 브라우저 안에서 시작할 수도 있습니다.

같은 화면에 세 겹을 입힙니다.

구조가 생기고, 모양이 정돈되고, 사용자의 행동에 반응합니다.

HTML

+ CSS

+ JavaScript

업무 요청 / 001

월간 실적 취합

담당자 민지 · 기한 9월 30일

검토할 요청

완료로 표시

미완료



HTML · 무엇이 있는가

제목, 문단, 입력란, 버튼처럼 화면의 구조와 의미를 정합니다.



CSS · 어떻게 보이는가

색, 글자, 간격, 배치로 읽는 순서를 만듭니다.



JavaScript · 무엇이 바뀌는가

버튼을 누르면 상태가 바뀌듯, 입력에 따라 화면과 데이터를 갱신합니다.

메모가 결과로 바뀌는 규칙.

백엔드는 요청을 받아, 업무 기준에 따라 처리하고 결과를 돌려줍니다.



① 받기

담당자·업무·기한을 읽습니다.

② 처리

빠진 값은 '확인 필요'로 남깁니다.

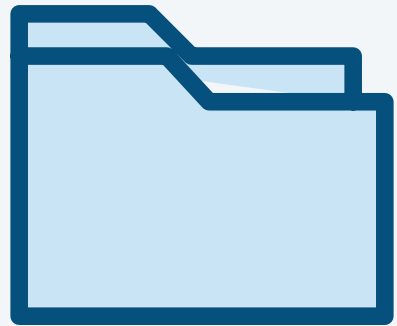
③ 반환

화면에 보여 줄 결과를 만듭니다.

Python은 이 처리를 구현할 수 있는 언어입니다. 첫 버전은 정해진 표기를 읽는 규칙 기반 처리입니다.

프로젝트 폴더는 시와 공유하는 작업대입니다.

코드, 참고 자료, 결과, 반복 규칙을 같은 맥락 안에서 다룹니다.



	input/	참고 자료
	app.py	처리 코드
	index.html	사용자 화면
	AGENTS.md	반복 규칙
	README.md	실행 안내

프로젝트 폴더 = 함께 쓰는 작업대

에이전트는 이 안의 코드와 자료를 읽고 필요한 부분을 수정합니다.

완료 메시지에서 실물로

어느 파일이 만들어졌는지, 무엇이 바뀌었는지, 실제 실행 결과를 함께 봅니다.

원본과 결과를 구분

입력 자료는 보존하고 가공한 결과는 별도 위치에 남깁니다.

터미널은 실행을 부탁하는 창.

터미널·셸·CLI는 서로 연결되지만 같은 뜻은 아닙니다.

터미널에 명령을 전달

```
python app.py
```



터미널

명령과 결과를 보는 창

터미널 → Python·개발도구 실행. 마우스 조작은 별도 GUI 자동화 도구의 역할입니다.

셸 · PowerShell 등

명령을 해석하는 프로그램

CLI

문자로 도구를 쓰는 방식

필요한 부품을 가져오고, 프로젝트별로 분리합니다.

라이브러리·패키지·패키지 관리자는 “재사용할 기능”을 준비하는 과정입니다.



프로젝트 A · 가상환경

프로젝트 B · 가상환경

라이브러리·패키지

가져다 쓰는 기능과 설치 묶음

pip · uv · npm

필요한 패키지를 준비하는 도구

가상환경

프로젝트별 Python 패키지를 분리

기록, 규칙, 기능, 연결. 비슷해 보여도 역할이 다릅니다.

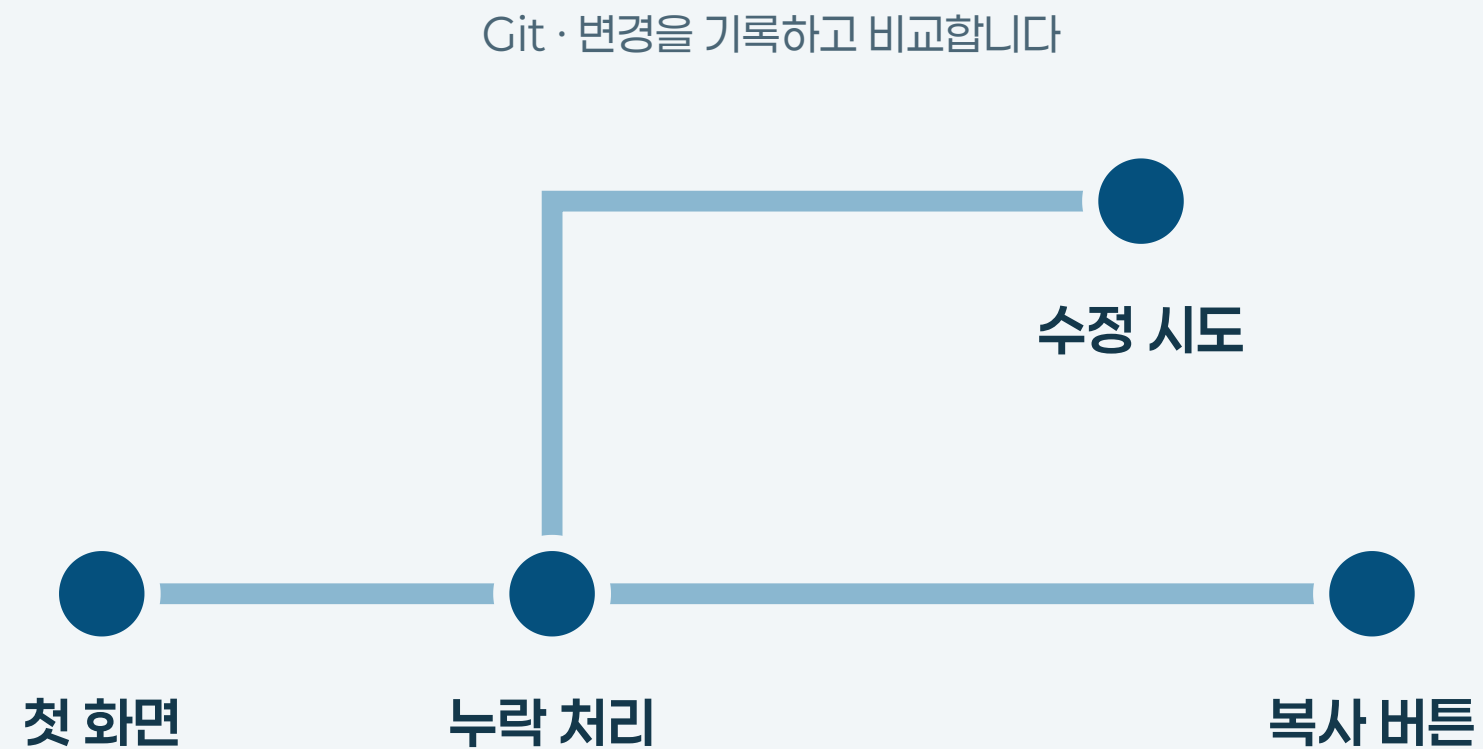
파일은 설명을 담고, 도구는 실제로 행동합니다.

	01		02		03		04
Markdown · .md		AGENTS.md		Skill		MCP	
제목·목록 등을 간단한 기호로 적는 문서 형식		이 프로젝트의 반복 작업 기준을 전달하는 문서		작업 방법과 참고 자료를 묶어 재사용하는 지침		AI 앱과 외부 도구·데이터를 잇는 공통 규약	
문서를 쓰는 형식		이곳의 작업 기준		일하는 방법		도구와 연결	

지침을 적었다고 권한이 생기지는 않습니다. 지침 적용 위치와 실제 도구 연결을 함께 확인합니다.

변경을 기억하는 도구, 설정을 전달하는 통로.

Git·GitHub와 환경변수는 서로 다른 문제를 해결합니다.



Git / GitHub

Git은 변경 이력 관리 도구, GitHub는 저장소를 온라인에서 보관하고 협업하는 서비스입니다.



환경변수

주소·인증 등의 설정을 코드 밖에서 전달합니다.

환경변수 자체가 자동으로 비밀값을 암호화하지는 않습니다.

모델을 고르는 기준도 결과에서 시작합니다.

작업의 난도·응답 속도·비용을 함께 보고 선택합니다.

같은 업무로 비교하기



수치는 성능 측정값이 아닌 비교 관점의 그림입니다.

기본값에서 작게 시작

간단한 변경과 여러 파일의 오류 분석은 필요한 검토 깊이가 다릅니다.

추론 노력 = 검토에 쓰는 자원

더 많은 시간·사용량을 쓸 수 있지만 정확성을 보장하지는 않습니다.

설정도 질문할 수 있습니다

“이 작업에 적합한 모델과 설정을 이유와 함께 설명해줘.”

이제 이 메모로 작은 도구를 만듭니다.

15분 제작 시연 · 파일 생성 → 실행 → 결과 확인 → 수정



첫 요청

업무 요청을 붙여 넣으면
담당자, 할 일, 기한을 정리해 주는
간단한 도구를 만들어줘.

외부 API 없이 첫 버전을 만듭니다.
실행 가능한 흐름부터 만들고 고칩니다.

개발화면은 네 곳만 봅니다.

어디에 생겼는지, 무엇이 바뀌었는지, 실제로 실행됐는지.

 화면 구성 예시		실제 배치는 설정에 따라 다릅니다VS Code
① 파일 app.py index.html AGENTS.md	② 코드와 변경 내용 def read_request(text): fields = parse_fields(text) return fields 에이전트가 수정한 위치를 확인합니다.	④ 에이전트 대화 “기한이 없으면 확인 필요로 표시해줘.” 파일 확인 → 수정 → 실행
	③ 터미널 python app.py 입력 1건 · 결과 저장 완료	

첫 버전은, 눈앞에서 작동하게.

붙여 넣은 값을 읽어 담당자·할 일·기한으로 정리합니다.



입력 · 가상 업무 자료

담당자: 민지

업무: 월간 실적 취합

기한: 9월 30일

- 기한 있음
- 기한 없음
- 빈 입력

담당자: / 업무: / 기한: 표기를 읽는 로컬 규칙입니다.
자유로운 문장의 의미를 이해하는 AI 기능은 아닙니다.

처리 결과

담당자	민지
할 일	월간 실적 취합
기한	9월 30일

입력한 세 항목을 정리했습니다.

결과 복사

빠진 기한을 어떻게 다룰까요?

오류가 사라졌는지보다 업무 규칙이 맞는지를 봅니다.

입력

담당자: 민지
업무: 월간 실적 취합



기한이 없는 요청도 들어옵니다.

수정 전

수정 후

기한	확인 필요
----	-------

누락을 숨기지 않고, 사람이 확인할 항목으로 남깁니다.

“기한이 없으면 추측하지 말고 ‘확인 필요’로 표시해줘.”

수정 뒤에는 기한이 있는 요청과 없는 요청을 모두 확인합니다. 새 규칙이 정상 입력까지 바꾸지 않는지 봅니다.

한 번 말한 기준을 다음 작업에도 이어갑니다.

AGENTS.md에는 반복 규칙을, 실행 안내에는 시작 방법을 남깁니다.

AGENTS.md · 예시

작업 규칙

- 화면은 한국어로 작성합니다.
- 기한이 없으면 확인 필요로 표시합니다.
- 수정 후 정상·누락·빈 입력을 확인합니다.
- 원본 자료는 덮어쓰지 않습니다.



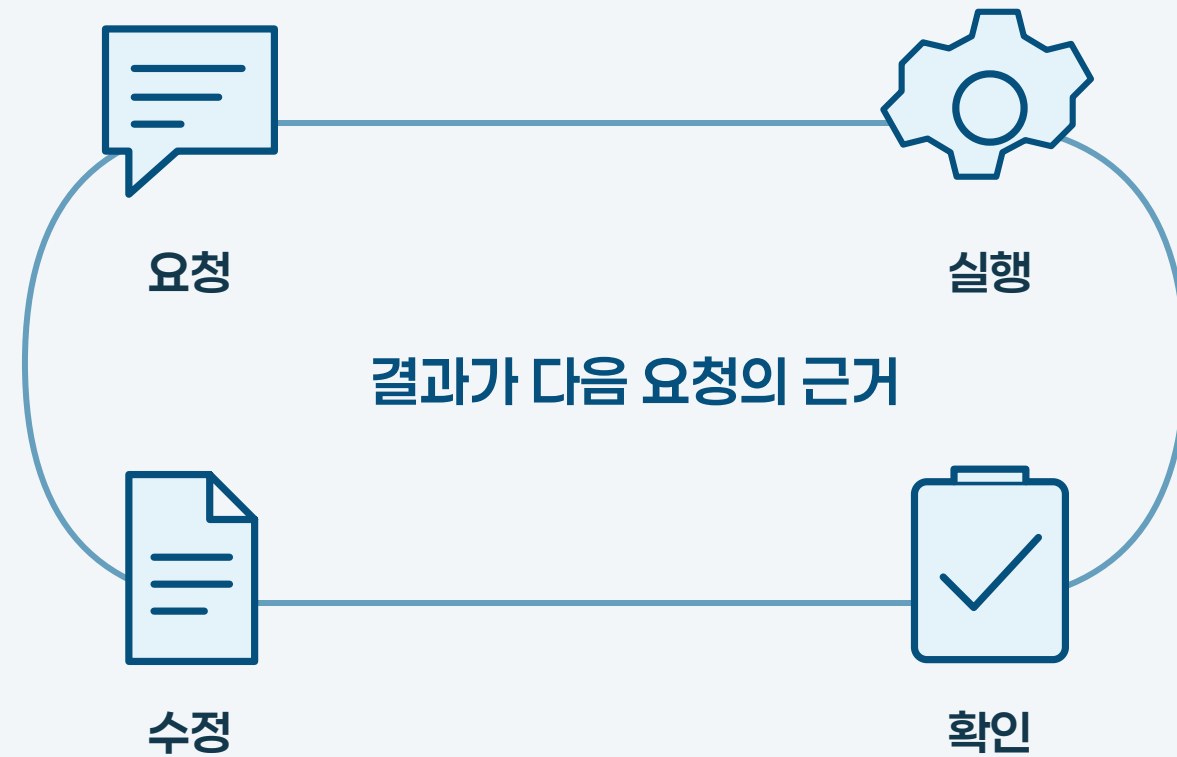
**앞으로도 이 기준을 따르도록
AGENTS.md에 정리해줘.**

직접 파일 문법을 외우지 않아도 됩니다. 반복해서 말하는 기준을 에이전트에게 정리해 달라고 요청합니다.

다른 컴퓨터에서도 실행할 수 있도록 필요한 패키지와 실행 명령을 README.md에 함께 남깁니다.

만드는 일은 한 번의 주문보다 짧은 왕복에 가깝습니다.

실행 결과가 다음 요청의 근거가 됩니다.



짧게 만들고,
실제로 확인합니다.

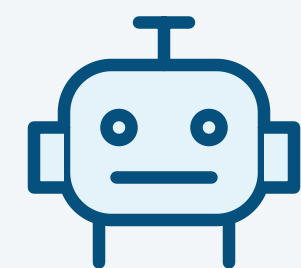
“기한이 없을 때 빈칸입니다.
확인 필요로 표시해줘.”

정상 입력과 예외 입력을 함께 확인합니다.

내 작업에 맞는 입구 하나.

앱·편집기 확장·CLI는 에이전트와 만나는 방식입니다.

앱



대화와 작업 공간

앱이 제공하는 작업 흐름에서 프로젝트를 다룹니다.

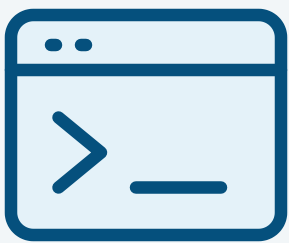
편집기 확장 · 시연 기준



파일과 대화를 함께

VS Code에서 코드, 터미널, 에이전트를 함께 봅니다.

CLI



명령어 기반 작업

터미널에서 에이전트를 실행하고 요청합니다.

Codex, Claude Code, Cline, Kilo Code 등은 도구 후보입니다. 실제 사용 형태·기능·접근 가능 여부는 사내 허용 환경에서 확인합니다.

무엇을 만들지에 따라 필요한 도구가 달라집니다.

모두 설치해야 시작할 수 있는 것은 아닙니다.

HTML 파일

Python 처리

npm 기반 웹 프로젝트



브라우저

이 예제의 실행에 사용



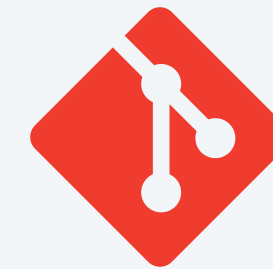
Python

작업에 따라 선택



Node.js

작업에 따라 선택



Git

이력 관리에 사용

단순 HTML 파일은 브라우저로 열 수 있습니다. 편집기는 파일을 고치는 도구이며, Python과 Node.js가 항상 필요한 것은 아닙니다.

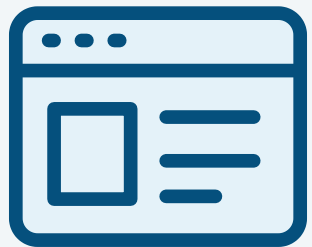
[VS Code](#) > [Python](#) > [Node.js](#) > [Git](#) >

공식 설치 안내 확인: 2026-09-17 · 회사에서 승인한 설치 경로를 우선합니다.

Git logo: Jason Long · CC BY 3.0. 각 제품의 공식 식별 아이콘을 교육 목적으로 사용.

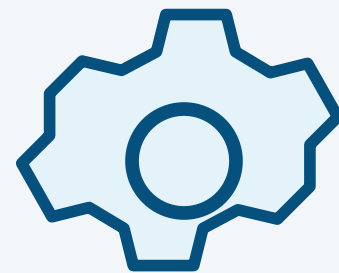
만들었다면, 실행 방법까지.

다른 컴퓨터에서는 파일 외에 실행 환경과 패키지도 필요할 수 있습니다.



HTML 파일

브라우저에서 열기. 단순 도구는 파일 하나로 사용할 수 있습니다.



Python 프로그램

Python과 필요한 패키지를 준비하고 명령어로 실행합니다.



서버가 있는 웹 앱

서버를 실행하거나 배포합니다. 화면 파일만 옮겨서는 부족합니다.

“처음 받는 사람도 실행할 수 있도록 설치 항목과 실행 순서를 적어줘.”

작업 범위와 승인 기준은 알고 말합니다.

접근할 수 있는 범위와 행동할 때 확인받는 방식은 구분합니다.



접근 범위

어떤 파일·명령·네트워크를 사용할 수 있는가.

승인 방식

어떤 행동 전에 사용자 확인이 필요한가.

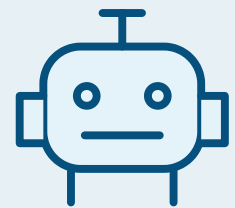
말길 일에 맞게 설정

Full Access가 모든 작업의 필수 조건은 아닙니다. 회사에서 허용한 도구와 권한을 확인합니다.

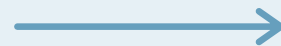
AI로 만든 프로그램과 AI를 부르는 프로그램.

개발할 때 AI를 썼다는 것과, 실행할 때 AI를 호출한다는 것은 다릅니다.

AI로 만든 프로그램



개발할 때 AI



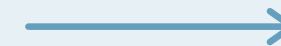
완성된 도구

실행할 때는 정해진 코드가 처리합니다.
첫 예제는 모델을 호출하지 않습니다.

AI를 부르는 프로그램



실행 중인 도구



API로 AI 호출

자유로운 문장을 AI에 보내 결과를 받습니다.
응답을 업무 기준으로 확인해야 합니다.

이 메모가 API로 건너갑니다.

프로그램이 AI 서비스에 요청하고, 응답을 받아 업무 화면에 사용합니다.



호출 구조를 설명하는 예시입니다. 실제 P-GPT 주소·인증·응답 형식은 사내 안내를 확인해야 합니다.

키는 화면 뒤에 듅니다.

API 키는 서비스 사용 권한을 확인하는 인증 정보입니다.

사용자에게 전달되는 화면

업무 요청 정리 도구

입력과 결과



API 키를 넣지 않음

서버 측 설정

백엔드



환경변수 등으로 인증 정보를 전달



HTML 안에 넣은 키는 이용자가 확인할 수 있습니다. 공개 저장소, 공유 문서, 로그에도 인증 정보를 남기지 않습니다.

공개 예제를 그대로 붙이기 전에.

사내 서비스의 실제 계약부터 확인합니다.



신청과 접근

신청 위치, 승인 담당, 이용 가능한 네트워크를 확인합니다.



주소와 인증

연결 주소, API 키 전달 방식, 허용 모델을 확인합니다.



요청과 응답

입력 형식, 결과 형식, 오류 응답과 지원 기능을 확인합니다.

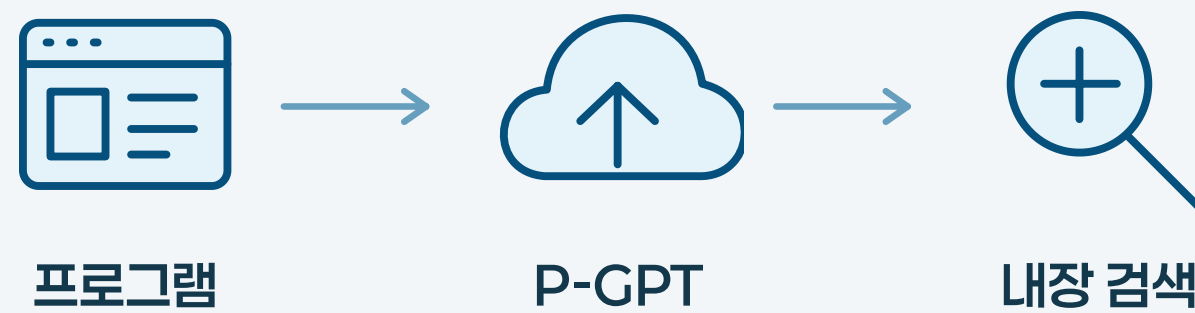
▼ Responses API 예제를 사용할 때

Responses API는 요청·응답 방식의 한 예입니다. P-GPT가 해당 방식과 필요한 기능을 지원하는지 확인한 뒤 실제 지원 형식에 맞춥니다. 공개 예제를 사내 연결 검증으로 간주하지 않습니다.

웹 검색도, 어디서 하느냐가 다릅니다.

AI 서비스 안의 검색과 프로그램의 직접 웹 접근을 구분합니다.

경로 A · 서비스 안에서 검색



P-GPT가 내장 검색을 제공하는지, 어떤 모델·도구에서 가능한지 확인합니다.

경로 B · 프로그램이 직접 접근



네트워크 허용 범위와 필요한 인증서·프록시 설정을 확인합니다.

접속·인증·인증서 오류는 원인이 다릅니다. 실제 오류와 발생 위치를 확인합니다.

내 업무에서는 무엇부터 바꿔볼까요?

개념, 시작 방법, 업무 적용에 대한 질문을 나눕니다.



반복해서 정리하는 자료가 있나요?



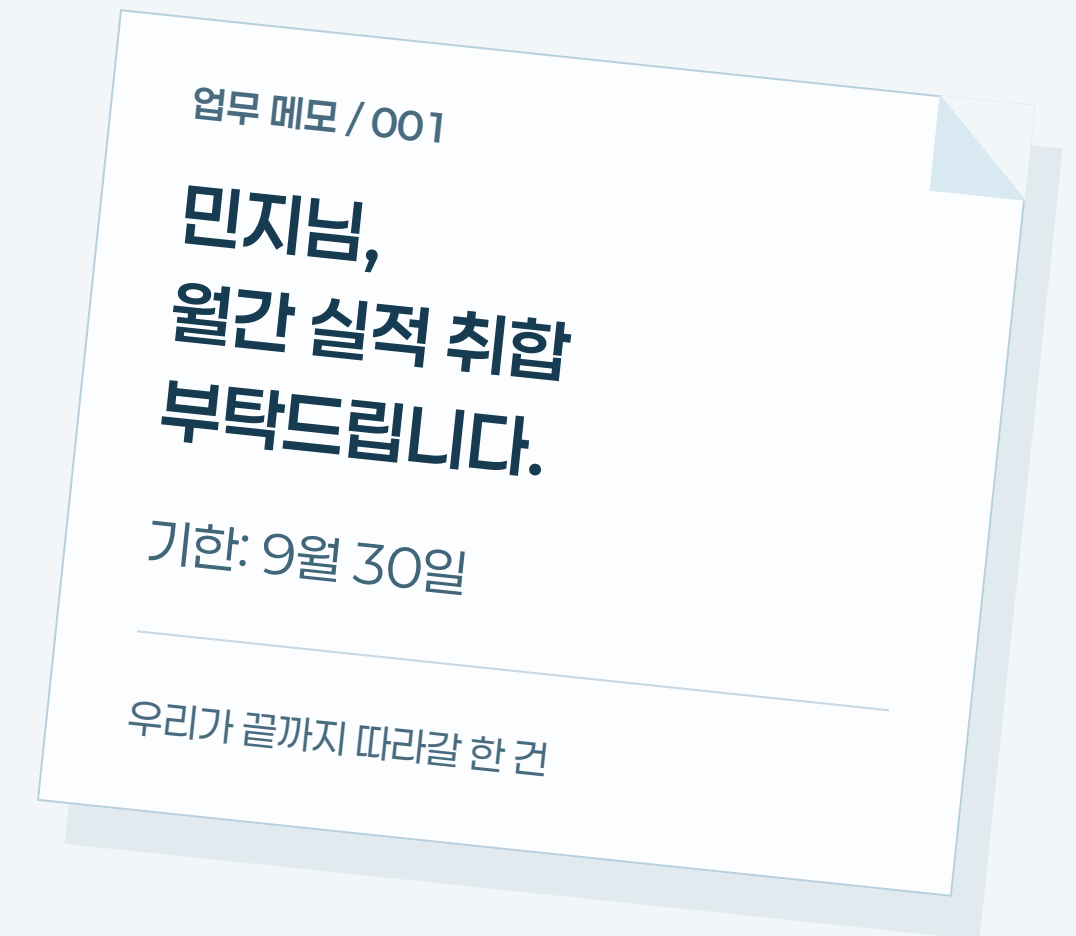
결과가 맞는지 판단할 기준은 무엇인가요?



가상 자료 한 건으로 어디까지 해볼 수 있나요?

모르는 것은 묻고.

필요한 것은 요청하고, 결과로 확인합니다.



필요할 때 다시 보는 개념

Vibe Coding

자연어로 의도를 전달하는 개발 방식

원하는 기능을 자연어로 요청하고, AI가 만든 프로그램을 실행·확인·수정하며 완성하는 방식입니다. 업무 담당자는 원하는 결과와 판단 기준을 설명합니다.

프론트엔드

Frontend · 사용자가 만나는 화면

브라우저의 제목, 목록, 입력란, 버튼과 그 동작을 담당합니다. 사용자의 입력을 받아 서버에 요청하거나 브라우저 안에서 직접 처리할 수 있습니다.

백엔드

Backend · 뒤에서 요청을 처리하는 부분

자료를 읽고 계산하고 저장한 뒤 결과를 돌려줍니다. Python은 구현에 사용할 수 있는 언어 중 하나입니다. 작은 파일 기반 도구에는 서버가 없어도 됩니다.

JavaScript

화면을 반응하게 만드는 프로그래밍 언어

버튼 클릭을 처리하고 목록을 필터링하거나 화면 내용을 바꿉니다. 브라우저 밖에서도 Node.js 같은 실행 환경으로 사용할 수 있습니다.

HTML

HyperText Markup Language · 화면의 구조

제목·문단·표·입력란처럼 화면을 구성하는 요소와 의미를 작성합니다. 이 발표 자료도 HTML로 구성돼 브라우저에서 열립니다.

CSS

Cascading Style Sheets · 화면의 모양

색·글자·간격·정렬·애니메이션을 정합니다. HTML의 같은 구조를 유지하면서 디자인을 바꿀 수 있습니다.

필요할 때 다시 보는 개념

CSV	Comma-Separated Values · 표를 저장하는 텍스트 행과 열로 된 자료를 쉼표로 구분해 저장합니다. 오늘 예제는 담당자·업무·상태 세 열을 사용합니다. 따옴표나 인코딩에 따라 읽기 처리가 필요할 수 있습니다.
Python	자료 처리와 자동화에 많이 쓰는 언어 자료 처리와 자동화를 구현할 수 있는 언어입니다. 이번 교육에서는 업무 요청을 읽고 정해진 규칙으로 결과를 만드는 역할을 설명합니다.
라이브러리	미리 만들어 둔 재사용 기능 표 처리·이미지 변환·차트 등 필요한 기능을 가져다 씁니다. 직접 작성할 코드를 줄이지만 버전과 사용 방법은 확인해야 합니다.
가상환경	프로젝트별 패키지를 분리하는 작업 환경 A 프로젝트와 B 프로젝트가 서로 다른 패키지 버전을 쓰도록 분리합니다. 별도의 컴퓨터나 가상 머신을 만드는 것은 아닙니다.
터미널	문자 명령을 입력하고 결과를 보는 창 터미널 안에서 PowerShell 같은 셸이 명령을 해석합니다. 파일 목록을 보거나 Python 프로그램을 실행하는 데 사용합니다.
CLI	Command Line Interface · 문자 명령 방식 버튼 대신 명령어와 옵션으로 프로그램을 조작합니다. 터미널은 창, 셸은 명령 해석기, CLI는 도구를 사용하는 방식입니다.

필요할 때 다시 보는 개념

PowerShell	Windows에서 사용하는 셸과 자동화 환경 명령을 해석하고 도구를 실행합니다. pwd는 현재 위치, dir은 목록, cd는 폴더 이동, cls는 화면 정리, ipconfig는 네트워크 설정 확인에 사용합니다.
GitHub	Git 저장소를 원격에 보관하는 협업 서비스 변경 이력을 공유하고 검토할 수 있습니다. 저장소가 공개되면 파일도 공개될 수 있으므로 코드에 API Key 등 비밀값을 넣지 않습니다.
Git	파일의 변경 이력을 관리하는 도구 작업의 특정 상태를 기록하고 차이를 비교하거나 이전 내용을 되찾을 수 있습니다. 커밋은 변경 상태를 남기는 기록 단위입니다.
환경변수	코드 밖에서 프로그램에 전달하는 설정 API 주소·모델 이름·인증 정보 등을 실행 환경에서 전달합니다. 그 자체가 자동 암호화 저장소는 아니므로 접근 권한과 로그 노출을 관리해야 합니다.
Node.js	브라우저 밖에서 JavaScript를 실행하는 환경 웹 서버나 개발 도구를 실행할 때 사용합니다. npm 기반 도구 설치 경로에서도 필요하지만 모든 Python 업무 자동화에 필요한 것은 아닙니다.
npm	JavaScript 패키지 관리 도구 필요한 패키지나 CLI를 설치하고 프로젝트 명령을 실행합니다. 설치 명령의 출처와 회사에서 허용한 경로를 확인합니다.

필요할 때 다시 보는 개념

Responses API	모델에 입력을 보내 응답을 받는 API 모델에 요청을 보내고 응답을 받는 API 방식의 하나입니다. 사내 P-GPT가 이 방식과 필요한 기능을 지원하는지 먼저 확인하고, 실제 지원되는 형식으로 요청해야 합니다.
API	Application Programming Interface · 프로그램 간 약속 어느 주소로 어떤 요청을 보내고 어떤 응답을 받을지 정합니다. 식당의 주문서처럼 형식이 맞아야 처리할 수 있습니다.
LLM	Large Language Model · 대규모 언어 모델 문맥을 바탕으로 언어를 생성·분석하는 모델입니다. 자연어 요약에는 유용하지만 정해진 규칙의 집계에 반드시 필요한 것은 아닙니다.
P-GPT	기획안에서 다루는 사내 AI 서비스 이번 교육에서 연결 대상으로 설명하는 사내 AI 서비스입니다. 실제 주소·인증·모델·요청 형식은 사내 공식 안내가 기준이며, 이 자료에서는 실연결을 검증하지 않았습니다.
AGENTS.md	프로젝트 작업 규칙을 전달하는 지침 파일 원본 보존·결과 폴더·오류 처리·검증 기준 같은 반복 규칙을 적습니다. 도구별로 읽는 위치와 적용 범위를 확인해야 합니다.
Full Access	Agent의 접근 범위를 넓히는 설정 파일·명령·네트워크 등의 접근 범위를 넓힐 수 있습니다. 사용 가능한 범위와 승인 방식은 환경에 따라 다르므로 회사 정책과 맡길 일을 함께 확인합니다.

필요할 때 다시 보는 개념

프록시	통신을 중간에서 전달하는 서버 사내 네트워크에서 외부 연결을 중계하거나 정책을 적용합니다. 주소가 맞아도 프록시 설정 때문에 접속이 되지 않을 수 있습니다.
CA	Certificate Authority · 인증서 발급 기관 통신 상대의 인증서를 신뢰할지 판단하는 체계와 관련됩니다. 회사에서 정한 CA 적용 방법을 따르고 검증을 끄는 방식으로 우회하지 않습니다.
추론	문제를 풀기 위해 검토하는 과정 문제를 풀기 위해 검토하는 과정입니다. 추론 노력의 지원 단계와 동작은 모델·도구마다 다릅니다. 많은 자원을 쓴다고 항상 정답인 것은 아닙니다.
VS Code	Visual Studio Code · 파일과 코드를 다루는 편집기 폴더·코드·터미널·확장을 한 화면에서 사용할 수 있습니다. AI 확장을 연결하면 프로젝트 맥락에서 작업을 요청할 수 있습니다.
DB	Database · 자료를 구조적으로 저장하는 곳 자료를 구조적으로 저장하고 조회·수정하는 데이터베이스입니다. 작은 도구는 파일에 결과를 저장하거나 브라우저 안에서만 처리할 수도 있어, 별도 DB가 항상 필요한 것은 아닙니다.
MCP	Model Context Protocol · 도구 연결 규약 AI 애플리케이션이 외부 도구나 데이터에 연결되는 공통 규약입니다. 어떤 기능을 쓸 수 있는지는 연결된 서버와 권한에 따라 달라집니다.

필요할 때 다시 보는 개념

Skill	반복 작업을 위한 지침과 자료 작업 순서, 참고 자료, 보조 스크립트 등을 묶어 재사용합니다. 실제로 읽고 적용하는 방식은 사용하는 에이전트에 따라 다릅니다.
셸	Shell · 명령을 해석하는 프로그램 터미널에 입력한 명령을 해석해 다른 프로그램을 실행합니다. PowerShell과 Bash가 대표적인 예입니다.
Markdown	마크다운 · 간단한 문서 작성 형식 #으로 제목을, -로 목록을 표현하는 등 텍스트에 간단한 기호를 붙여 문서를 작성합니다. .md는 흔히 사용하는 파일 확장자입니다.
패키지	설치와 배포를 위한 묶음 재사용할 코드와 관련 자료를 설치 가능한 단위로 묶습니다. 라이브러리를 패키지로 배포할 수 있습니다.
pip	Python 패키지 설치 도구 Python 프로젝트에서 필요한 패키지를 설치하고 관리할 때 사용합니다. 프로젝트의 가상환경 안에서 실행할 수 있습니다.
uv	Python 프로젝트와 패키지 관리 도구 Python 환경과 패키지를 준비하고 프로젝트를 실행하는 작업을 돕습니다. pip과 모든 명령이 같은 것은 아닙니다.